

BUILDER JOURNAL

Week 2 Reflection

FELLOW	TRACK	PROGRAM
M. Abdullah Nazir	Computer Vision 26	BS AI

SYSTEM PERFORMANCE SUMMARY

METRIC	VALUE / CONFIGURATION DETAIL
Final mAP50	0.871 (87.1%)
Final Precision	0.935 (93.5%)
Training Timeline	59 minutes 50 epochs on Custom Environment
Dataset Scale	2,037 images (via Roboflow Universe)

"Week 2 in one line: I built a working bottle detection system from zero, debugged real errors, ran four experiments, and learned more from the problems than from the successes."

APPLICATIONS & ENGINEERING CHALLENGES

Building this project required solving key obstacles covering data ingestion, framework configurations, and multiprocessing issues during network training runs:

01/ DATASET · DATASET CHALLENGES

The Challenge: Manual data collection would be too slow and the resulting dataset would lack diversity — all photos taken in the same environment with similar lighting. My first instinct was to search my own surroundings and take photos with my phone, but collecting 300+ diverse images manually would take an entire day.

THE SOLUTION · ROBOFLOW UNIVERSE INTEGRATION

I discovered Roboflow Universe and integrated a bottle detection dataset (breathless/bottle-ewqkq, Version 8) with 2,037 augmented images (1,782 train / 169 valid / 86 test). This gave me far more diverse data than I could have collected manually, covering various types, backgrounds, and occlusions.

02/ ANNOTATION · ANNOTATION & CONFIG CHALLENGES

The Challenge: Configuring the `data.yaml` file correctly. The downloaded dataset had a path error — the `yaml` file contained backslash-forward-slash mixed paths like `/valid` instead of the correct folder name sequence on my system.

THE SOLUTION · PATH RE-ALIGNMENT

I manually edited `data.yaml` to use forward slashes throughout and verified the `val` folder name matched exactly what was on disk. The final configuration used explicit full absolute paths to `train`, `valid`, and `test` folders, which eliminated any layout resolution ambiguity.

03/ TRAINING · WINDOWS MULTIPROCESSING CRASH

The Challenge: Windows multiprocessing crash. When I first ran `train.py`, Python immediately crashed with a `RuntimeError` about spawning processes: *'An attempt has been made to start a new process before the current process has finished bootstrapping.'*

THE SOLUTION · MAIN GUARD REALIGNMENT

Python on Windows requires that training code be wrapped inside an `if __name__ == '__main__':` guard. This tells Windows that this is the main process and it is safe to start child processes for data loading. Adding this one line immediately resolved the environment crash.

04/ RUNTIME · EXPERIMENT NESTING ERROR

The Challenge: After training with 25 epochs, the script could not find the saved model. It looked in `runs/experiments/` but YOLO actually saved to `runs/detect/runs/experiments/` due to nested tracking paths.

THE SOLUTION · ABSOLUTE TRACKING PATHS

I wrote a fixed version of the experiment script that uses absolute paths for both training output and model loading, recovering the 25-epoch intermediate benchmarks (mAP50: 0.860, Precision: 0.906) securely.

WHAT I LEARNED THIS WEEK (OPTIMIZATION MATRIX)

IMPROVEMENT VECTOR	BEFORE FIX STATE	AFTER OPTIMIZATION STATE
Windows Multi-threading	Script crashed immediately on launch	Training runs successfully with processes
Path Resolution for Experiments	FileNotFoundError on model lookup	Model loads and tracks correctly
Confidence Thresholding	Default 0.25 (arbitrary layout)	0.40 optimized from Experiment 3 analysis
data.yaml Configuration Paths	Mixed slashes causing processing error	Clean forward slashes running seamlessly

CORE TAKEAWAYS & LESSONS LEARNED

Through empirical tracking, five critical foundational blocks were stabilized this week:

- **Public Datasets as Assets:** Open-source data repositories are standard industrial tools used to bootstrap projects before deploying targeted validation checks.
- **Detailed Error Parsing:** Both runtime crash traces contained descriptive strings that directed explicitly to the solution parameters when parsed thoroughly.
- **Sidebar Optimization Thresholds:** Tuning the confidence threshold to 0.40 yielded an optimal balance of 93.4% stable Precision instead of relying on standard defaults.
- **Transfer Learning Efficiencies:** Initializing from COCO weights enabled high-level feature parsing in under 59 minutes, which would take days from scratch.
- **Model Scaling Laws:** YOLOv8 Nano (3M parameters) delivered 87.1% mAP50, confirming small architectures can perform Single-Class tasks with excellent resource efficiency.

WEEK 3 STRATEGIC MILESTONES

#	STRATEGIC GOAL	DETAIL & ROADMAP TARGET
1	Deep Learning Fundamentals	Study CNN architectures, backpropagation math, and feature loss functions in detail.
2	Multi-class Detection	Extend the current bottle system to detect multiple variant item classes simultaneously.
3	Improve Recall Score	Current Recall is 79.2% — target an expansion to 85%+ using targeted occlusion training.
4	Real-time Video Pipeline	Build a continuous streaming webcam detection loop targeting a stable 25+ FPS.
5	Model Comparison Study	Run an empirical evaluation benchmark comparing YOLOv8 against a Faster R-CNN backend.
6	GitHub Portfolio Polish	Write professional documentation README layouts and showcase inference outputs.